Міністерство освіти і науки України
Центральноукраїнський національний технічний університет
Механіко-технологічний факультет
Кафедра кібербезпеки та програмного забезпечення
Дисципліна: Базові методології та технології програмування

Лабораторна робота №9

Тема: «Реалізація програмних модулів розгалужених та ітераційних обчислювальних процесів»

Виконав: ст. гр. КН-25
Грицюк Є.В.
Перевірив: викладач
Коваленко А.С.

Кропивницький 2026

ТЕМА: Реалізація програмних модулів розгалужених та ітераційних обчислювальних процесів

МЕТА: полягає у набутті ґрунтовних вмінь і практичних навичок реалізації технології модульного програмування, застосування операторів C/C++ арифметичних, логічних, побітових операцій, умови, циклів та вибору під час розроблення статичних бібліотек, заголовкових файлів та програмних засобів у кросплатформовому середовищі Code::Blocks.

ЗАВДАННЯ: 1. Реалізувати функції розв'язування задач 9.1–9.3 як складових статичної бібліотеки libModulesПрізвище.a (проект ModulesПрізвище, створений під час виконання лабораторної роботи №8). 2. Реалізувати програмне забезпечення розв'язування задачі 9.4 на основі функцій статичної бібліотеки libModulesПрізвище.a.

Варіант 4

Аналіз і постановка задачі 9.1

Умова: Залежно від введеної суми покупки розрахувати відсоток знижки згідно із заданими діапазонами, обчислити абсолютний розмір знижки у гривнях та кінцеву суму до сплати (якщо сума понад 1000 грн, сплачується 75%, отже, знижка становить 25%). Вхідні дані: дійсне число purchase_amount (сума покупки). Вихідні дані: числове значення discount_percent (відсоток знижки), дійсне число discount_uah (розмір знижки у гривнях) та дійсне число final_amount (сума до сплати). Обмеження (ОДЗ): значення purchase_amount має бути невід'ємним (purchase_amount >= 0).

Аналіз і постановка задачі 9.2

Умова: За введеним розміром чоловічої білизни в українській системі визначити та вивести відповідні розміри в міжнародній системі, системі Великобританії та Франції на основі наведеної таблиці відповідності. Вхідні дані: ціле число ukr_size (український розмір). Вихідні дані: текстові або числові повідомлення з відповідниками розмірів для міжнародної системи (intl_size), Великобританії (uk_size) та Франції (fr_size). Обмеження (ОДЗ): значення ukr_size має належати дискретній множині значень із таблиці: {44, 46, 48, 50, 52, 54}.

Аналіз і постановка задачі 9.3

Умова: Для введеного числа N перевірити значення біта D. Якщо він дорівнює 1, обчислити та вивести кількість двійкових нулів у числі N, інакше — обчислити та вивести кількість двійкових одиниць. Вхідні дані: ціле невід'ємне число N. Вихідні дані: ціле число bit_count (кількість знайдених нулів або одиниць). Обмеження (ОДЗ): значення N має належати діапазону від 0 до 190550 включно.

Аналіз вимог, проектування архітектури та детальне проектування програмного модуля.

Функціональні вимоги:

- Модуль 9.1: Повинен приймати дійсне число (суму покупки) і за допомогою багато альтернативного розгалуження (if - else if) визначати відсоток знижки. Повинен обчислювати грошовий еквівалент знижки та фінальну суму до сплати.
- Модуль 9.2: Повинен приймати ціле число (український розмір) і за допомогою оператора множинного вибору (switch-case) визначати відповідники у трьох інших системах.
- Модуль 9.3: Повинен приймати натуральне число N. Повинен виділяти 3-й біт. Повинен використовувати цикл для проходу по бітах числа та тернарний оператор ? : для інкрементації лічильника нулів або одиниць залежно від стану біта D.

Нефункціональні вимоги:

- Кожна задача має бути реалізована як незалежна функція (модуль), що не містить операторів введення з клавіатури (лише отримання аргументів та повернення результату).
- Типи даних мають бути оптимізовані: double для фінансових розрахунків (9.1), int для розмірів одягу (9.2) та бітових операцій (9.3).

Проектування архітектури:

Програмне забезпечення проектується за модульним принципом. Інтерфейс (прототипи) функцій розв'язання задач 9.1–9.3 буде розміщено у заголовковому файлі ModulesHrytsiuk.h, а їхня реалізація — у файлі ModulesHrytsiuk.cpp.

Головна програма, що міститься у файлі main.cpp, відповідатиме виключно за організацію діалогу з користувачем (введення вхідних даних), виклик функцій з підключеної статичної бібліотеки та форматоване виведення результатів на екран.

Визначено такі прототипи функцій:

1. int task9_1_discount(double purchase_amount, double &discount_uah, double &final_amount);
2. std::string task9_2_size_converter(int ukr_size);
3. int task9_3_bit_counter(int N);

Детальне проектування архітектури:

Модуль 1 (Задача 9.1): Обчислення знижки

- Сигнатура: int task9_1_discount(double purchase_amount, double &discount_uah, double &final_amount);
- Алгоритм:
    1. Перевірити ОДЗ: якщо purchase_amount < 0, завершити роботу функції (або встановити нульові значення).
    2. Ініціалізувати локальну змінну percent (відсоток знижки).
    3. Використовуючи багато альтернативне розгалуження if - else if, визначити значення percent залежно від діапазону

purchase_amount (від 5% для суми < 100 до 25% для суми > 1000).

4. Обчислити абсолютний розмір знижки: discount_uah = purchase_amount * (percent / 100.0). Результат записується за посиланням.
5. Обчислити суму до сплати: final_amount = purchase_amount - discount_uah. Результат записується за посиланням.
6. Повернути ціле значення percent через return.

Модуль 2 (Задача 9.2): Конвертер розмірів білизни

- Сигнатура: std::string task9_2_size_converter(int ukr_size);
- Алгоритм:
    1. Використати оператор множинного вибору switch (ukr_size).
    2. Для кожного валідного значення (case 44:, case 46:, case 48:, case 50:, case 52:, case 54:) сформувати та повернути рядок (std::string), що містить відповідні розміри у міжнародній системі, системі Великобританії та Франції згідно з таблицею ОДЗ.
    3. У гілці default: повернути повідомлення про те, що такого розміру не існує (обробка некоректного вводу).

Модуль 3 (Задача 9.3): Бітовий аналізатор

- Сигнатура: int task9_3_bit_counter(int N);
- Алгоритм:
    1. Отримати натуральне число N.
    2. Визначити стан біта D за допомогою побітового "І" з маскою: bool d3_is_one = (N & (1 << 3)) != 0;.
    3. Ініціалізувати лічильник count = 0.
    4. Запустити цикл while (N > 0) для почергового перегляду бітів.
    5. На кожній ітерації отримувати молодший біт: int current_bit = N & 1;.
    6. Використати тернарний оператор для підрахунку згідно з умовою:
    count += d3_is_one ? (current_bit == 0 ? 1 : 0) : (current_bit == 1 ? 1 : 0);

7. Виконати побітовий зсув числа N вправо на 1 розряд (N >>= 1).
8. Після завершення циклу повернути значення count.

Artifact: Test Suite

Date: DATE \@ "M/d/yyyy"
10/22/2024

Назва тестового набору Test Suite Description	TS_9_1
Назва проекта / ПЗ Name of Project / Software	Hrytsiuk-task_9_1
Рівень тестуванн Level of Testin	Модульне тестування
Автор тестсьют Test Suite Autho	Грицюк Євгеній
Виконавец Implemente	Грицюк Євгеній

Ід-р тест-кейса / Test Case ID	Дії (кроки) / Action (Test Steps)	Очікуваний результат / Expected Result	Результат тестування / Test Result
TC-01	purchase_amount = 50.0 discount_uah = 0.0 final_amount = 0.0	Повертає: 5 Змінює: discount_uah = 2.5, final_amount = 47.5	passed
TC-02	purchase_amount = 100.0 discount_uah = 0.0 final_amount = 0.0	Повертає: 7 Змінює: discount_uah = 7.0, final_amount = 93.0	passed
TC-03	purchase_amount = 350.0 discount_uah = 0.0 final_amount = 0.0	Повертає: 12 Змінює: discount_uah = 42.0, final_amount = 308.0	passed

Artifact: Test Suite

Date: DATE \@ "M/d/yyyy"
10/22/2024

Назва тестового набору Test Suite Description	TS_9_2
Назва проекта / ПЗ Name of Project / Software	Hrytsiuk-task_9_2
Рівень тестуванн Level of Testin	Модульне тестування
Автор тестсьют Test Suite Autho	Грицюк Євгеній
Виконавец Implemente	Грицюк Євгеній

Ід-р тест-кейса / Test Case ID	Дії (кроки) / Action (Test Steps)	Очікуваний результат / Expected Result	Результат тестування / Test Result
TC-01	ukr_size = 46	"Міжнародний: M, UK: 36, Франція: 3"	passed
TC-02	ukr_size = 45	"Розмір не знайдено"	passed
TC-03	ukr_size = 54	"Міжнародний: XXXL, UK: 44, Франція: -"	passed

Назва тестового набору Test Suite Description	TS_9_3
Назва проекта / ПЗ Name of Project / Software	Hrytsiuk-task_9_3
Рівень тестуванн Level of Testin	Модульне тестування
Автор тестсьют Test Suite Autho	Грицюк Євгеній
Виконавец Implemente	Грицюк Євгеній

Ід-р тест-кейса / Test Case ID	Дії (кроки) / Action (Test Steps)	Очікуваний результат / Expected Result	Результат тестування / Test Result
TC-01	N = 11	1	passed
TC-02	N = 5	2	passed
TC-03	N = 8	3	passed

Лістинг програми

```
#include <cmath>
#include <stdexcept>
#include <string>

constexpr double PI = 3.14159265358979323846;

double s_calculation(double x, double y, double z) {
    if (z < 0.0) {
        throw std::invalid_argument("z не може бути від'ємним");
    }
    double term1 = pow(x + y + z, 2.0);
    double term2 = 2.0 * PI * sqrt(z / 3.0);
    return term1 - term2;
}

int task9_1_discount(double purchase_amount, double &discount_uah, double
&final_amount) {
    if (purchase_amount < 0) {
        discount_uah = 0;
        final_amount = 0;
        return 0;
    }

    int percent = 0;
    if (purchase_amount < 100) {
        percent = 5;
    } else if (purchase_amount < 200) {
        percent = 7;
    } else if (purchase_amount < 300) {
        percent = 9;
    } else if (purchase_amount < 400) {
        percent = 12;
    } else if (purchase_amount < 500) {
        percent = 15;
```

```
    } else if (purchase_amount <= 1000) {
        percent = 20;
    } else {
        percent = 25;
    }

    discount_uah = (purchase_amount * percent) / 100.0;
    final_amount = purchase_amount - discount_uah;

    return percent;
}


std::string task9_2_size_converter(int ukr_size) {
    switch (ukr_size) {
        case 44: return "Міжнародний: S, UK: 34, Франція: 2";
        case 46: return "Міжнародний: M, UK: 36, Франція: 3";
        case 48: return "Міжнародний: L, UK: 38, Франція: 4";
        case 50: return "Міжнародний: XL, UK: 40, Франція: 5";
        case 52: return "Міжнародний: XXL, UK: 42, Франція: 6";
        case 54: return "Міжнародний: XXXL, UK: 44, Франція: -";
        default: return "Розмір не знайдено";
    }
}


int task9_3_bit_counter(int N) {
    if (N < 0) return 0;

    bool d3_is_one = (N & (1 << 3)) != 0;

    int count = 0;
    int temp = N;

    while (temp > 0) {
        int current_bit = temp & 1;

        count += d3_is_one ? (current_bit == 0 ? 1 : 0) : (current_bit == 1 ?
1 : 0);

        temp >>= 1;
    }

    return count;
}
```

Лістинг TestDriver

```
#include <iostream>
#include <string>
#include <iomanip>
#include <Windows.h>

#include "ModulesHrytsiuk.h"

using namespace std;

double my_abs(double val) {
    return val < 0.0 ? -val : val;
}
```

```
void print_test_result(string test_name, bool passed) {
    cout << left << setw(15) << test_name << ": ";
    if (passed) {
        cout << "[PASS]" << endl;
    } else {
        cout << "[FAILED]" << endl;
    }
}

void run_task9_1_tests() {
    cout << "--- Тестування задачі 9.1 (Знижки) ---" << endl;
    double d_uah, f_amt;
    int percent;

    percent = task9_1_discount(50.0, d_uah, f_amt);
    print_test_result("TC-01", (percent == 5 && my_abs(d_uah - 2.5) < 0.01 &&
my_abs(f_amt - 47.5) < 0.01));

    percent = task9_1_discount(100.0, d_uah, f_amt);
    print_test_result("TC-02", (percent == 7 && my_abs(d_uah - 7.0) < 0.01 &&
my_abs(f_amt - 93.0) < 0.01));

    percent = task9_1_discount(350.0, d_uah, f_amt);
    print_test_result("TC-03", (percent == 12 && my_abs(d_uah - 42.0) < 0.01
&& my_abs(f_amt - 308.0) < 0.01));
    cout << endl;
}

void run_task9_2_tests() {
    cout << "--- Тестування задачі 9.2 (Розміри) ---" << endl;

    string res1 = task9_2_size_converter(46);
    print_test_result("TC-01", (res1 == "Міжнародний: M, UK: 36, Франція:
3"));

    string res2 = task9_2_size_converter(45);
    print_test_result("TC-02", (res2 == "Розмір не знайдено"));

    string res3 = task9_2_size_converter(54);
    print_test_result("TC-03", (res3 == "Міжнародний: XXXL, UK: 44, Франція:
-"));
    cout << endl;
}

void run_task9_3_tests() {
    cout << "--- Тестування задачі 9.3 (Біти) ---" << endl;

    print_test_result("TC-01", (task9_3_bit_counter(11) == 1));
    print_test_result("TC-02", (task9_3_bit_counter(5) == 2));
    print_test_result("TC-03", (task9_3_bit_counter(8) == 3));
    cout << endl;
}

int main() {
    SetConsoleOutputCP(1251);
    SetConsoleCP(1251);

    cout << "ЗАПУСК МОДУЛЬНИХ ТЕСТІВ (UNIT TESTS)\n" << string(40, '=') <<
"\n\n";
```

```
    run_task9_1_tests();
    run_task9_2_tests();
    run_task9_3_tests();

    cout << string(40, '=') << "\nТестування завершено.\n";

    system("pause");
    return 0;
}
```

```
ЗАПУСК МОДУЛЬНИХ ТЕСТІВ (UNIT TESTS)
========================================

--- Тестування задачі 9.1 (Знижки) ---
TC-01          : [PASS]
TC-02          : [PASS]
TC-03          : [PASS]

--- Тестування задачі 9.2 (Розміри) ---
TC-01          : [PASS]
TC-02          : [PASS]
TC-03          : [PASS]

--- Тестування задачі 9.3 (Біти) ---
TC-01          : [PASS]
TC-02          : [PASS]
TC-03          : [PASS]

========================================
Тестування завершено.
```

Аналіз і постановка задачі 9.4

Умова: Розробити консольний додаток, який реалізує діалогове меню для виклику функцій з розробленої статичної бібліотеки. За введенням символу ‘s’ викликається функція s_calculation(), ‘f’ – функція задачі 9.1, ‘g’ – функція задачі 9.2, ‘h’ – функція задачі 9.3. Некоректні символи ігноруються з видачею звукового сигналу про помилкове введення. Після виконання обраної дії програма запитує користувача про продовження роботи; якщо введено символ ‘J’ або ‘j’, відбувається вихід з програми, інакше — цикл повторюється. Вхідні дані: символьна змінна menu_choice (вибір пункту меню), символьна змінна exit_choice (вибір для виходу з

програми), а також набір вхідних даних, які потрібні для конкретно обраної функції. Вихідні дані: форматовані текстові повідомлення в консолі (вивід меню, результати розрахунків обраної функції), звуковий сигнал (\a), що свідчить про помилку введення. Обмеження (ОДЗ): програма повинна працювати у безкінечному циклі доти, доки користувач явно не введе умову виходу (‘J’ або ‘j’).

Функціональні вимоги:

- Відображення текстового меню з доступними опціями.
- Зчитування символу для вибору дії.
- Використання оператора множинного вибору switch-case для маршрутизації викликів функцій залежно від введеного символу ('s', 'f', 'g', 'h').
- Обробка некоректного вводу в гілці default оператора switch із відтворенням звукового сигналу (за допомогою керуючого символу \a).
- Виклик розроблених модулів з підключеної статичної бібліотеки та відображення повернутих результатів.
- Організація циклічного виконання програми за допомогою циклу (наприклад, do-while або while(true)).

Нефункціональні вимоги:

- Додаток повинен бути розроблений як окремий проект, який лінкується зі статичною бібліотекою libModulesHrytsiuk.a.
- Підключення заголовкового файлу "ModulesHrytsiuk.h".

Проектування архітектури:

Програмне забезпечення реалізується у вигляді консольного клієнтського додатка, що містить єдину головну функцію main(). Цей додаток не містить власної логіки розрахунків, а виступає лише як інтерфейс користувача. Уся обчислювальна логіка інкапсульована у підключеній статичній бібліотеці. Архітектура програми побудована навколо головного циклу обробки подій, який чекає на ввід користувача, делегує задачу відповідному модулю та повертається на початок, поки не отримає команду на завершення.

Детальне проектування архітектури:

Почати безкінечний цикл while (true).
Вивести на екран текстове меню з переліком доступних команд:

- s — Розрахунок формули
- f — Обчислення знижки (Задача 9.1)
- g — Конвертер розмірів (Задача 9.2)
- h — Бітовий аналізатор (Задача 9.3)

Зчитати значення у символьну змінну menu_choice.
Використати оператор switch (menu_choice):

- case 's': Запросити ввід x, y, z. Викликати s_calculation(). Вивести результат. Перервати switch (break).
- case 'f': Запросити ввід amount. Викликати task9_1_discount(). Вивести відсоток, суму знижки та фінальну суму. Перервати switch.
- case 'g': Запросити ввід size. Викликати task9_2_size_converter(). Вивести результат. Перервати switch.
- case 'h': Запросити ввід N. Викликати task9_3_bit_counter(). Вивести результат. Перервати switch.
- default: Вивести керуючий символ \a (звуковий сигнал beep) та повідомлення про помилку вводу. Перервати switch.

Вивести повідомлення: "Бажаєте завершити роботу? (Введіть J або j для виходу, будь-який інший символ для продовження):".
Зчитати значення у символьну змінну exit_choice.
Якщо exit_choice == 'J' АБО exit_choice == 'j', перервати головний цикл (break).
Кінець циклу while.
Завершити роботу програми (return 0).

Назва тестового набору Test Suite Description	TS_9_4
Назва проекта / ПЗ Name of Project / Software	Hrytsiuk-task_9_4
Рівень тестуванн Level of Testin	Системне тестування
Автор тестсьют Test Suite Autho	Грицюк Євгеній
Виконавец Implemente	Грицюк Євгеній

Ід-р тест-кейса / Test Case ID	Дії (кроки) / Action (Test Steps)	Очікуваний результат / Expected Result	Результат тестування / Test Result
TC-01	1. Запустити програму. 2. У головному меню ввести символ f. 3. Ввести суму 100. 4. На запит про вихід ввести символ n	Програма викликає модуль розрахунку знижки, виводить результати (7%, 7 грн, 93 грн). Після введення n програма не закривається, а знову виводить головне меню.	passed
TC-02	1. Знаходячись у головному меню, ввести символ g. 2. Ввести розмір 46. 3. На запит про вихід ввести Enter (або будь-яку іншу літеру, крім J/j).	Програма відображає міжнародний розмір M. Після відповіді на запит виходу цикл повторюється, з'являється головне меню.	passed
TC-03	1. У головному меню ввести символ s. 2. Ввести дані: x=1, y=2, z=3. 3. На запит про вихід ввести 1.	Програма викликає функцію s_calculation(), виводить результат розрахунку формули, запитує про вихід і повертається в меню.	passed
TC-04	1. У головному меню ввести некоректний символ w (або цифру 5).	Програма ігнорує виклик обчислювальних модулів, виводить повідомлення про помилку введення, відтворює системний звуковий сигнал (beep) і переходить до запиту про завершення роботи.	passed
TC-05	1. У головному меню ввести символ h. 2. Ввести число 11. 3. На запит про завершення роботи ввести маленьку літеру j.	Програма коректно підраховує біти (1), після чого (через введення j) виходить із безкінечного циклу і повністю завершує свою роботу.	passed

TC-06	1. Запустити програму. 2. Навмисно викликати помилку (ввести x). 3. На запит про завершення роботи ввести велику літеру J.	Програма відтворює звук помилки, запитує про вихід, розпізнає велику літеру J і штатно завершує роботу.	passed

Лістинг

```
#include <iostream>
#include <string>
#include <Windows.h>


#include "ModulesHrytsiuk.h"

using namespace std;

int main() {
    SetConsoleOutputCP(1251);
    SetConsoleCP(1251);

    char menu_choice;
    char exit_choice;

    while (true) {
        cout << "\n================ ГОЛОВНЕ МЕНЮ ================\n";
        cout << "s - Обчислення функції s_calculation\n";
        cout << "f - Задача 9.1 (Обчислення знижки)\n";
        cout << "g - Задача 9.2 (Конвертер розмірів білизни)\n";
        cout << "h - Задача 9.3 (Бітовий аналізатор)\n";
        cout << "==============================================\n";
        cout << "Оберіть дію: ";
        cin >> menu_choice;

        switch (menu_choice) {
            case 's': {
                double x, y, z;
                cout << "Введіть значення x, y, z (через пробіл): ";
                cin >> x >> y >> z;
                try {
                    double result = s_calculation(x, y, z);
                    cout << "Результат s_calculation: " << result << endl;
                } catch (const exception& e) {
                    cout << "Помилка: " << e.what() << endl;
                }
                break;
            }
            case 'f': {
                double amount, d_uah = 0, f_amt = 0;
```

```
                cout << "Введіть суму покупки (грн): ";
                cin >> amount;
                int percent = task9_1_discount(amount, d_uah, f_amt);
                if (percent == 0 && amount < 0) {
                    cout << "Помилка: Сума не може бути від'ємною!\n";
                } else {
                    cout << "Відсоток знижки: " << percent << "%\n";
                    cout << "Сума знижки: " << d_uah << " грн\n";
                    cout << "Сума до сплати: " << f_amt << " грн\n";
                }
                break;
            }
            case 'g': {
                int size;
                cout << "Введіть український розмір чоловічої білизни: ";
                cin >> size;
                string result = task9_2_size_converter(size);
                cout << "Результат: " << result << endl;
                break;
            }
            case 'h': {
                int n;
                cout << "Введіть натуральне число N (від 0 до 190550): ";
                cin >> n;
                int count = task9_3_bit_counter(n);
                cout << "Результат підрахунку бітів: " << count << endl;
                break;
            }
            default:
                cout << "\a";
                cout << "Помилка: Невідома команда! Символ зігноровано.\n";
                break;
        }

        cout << "\nБажаєте завершити роботу? (Введіть J або j для виходу,
будь-який інший символ для продовження): ";
        cin >> exit_choice;

        if (exit_choice == 'J' || exit_choice == 'j') {
            cout << "Завершення роботи програми..." << endl;
            break;
        }
    }

    return 0;
}
```

Аргументи:

1. Практичне засвоєння принципу модульного програмування, що є

стандартом у розробці серйозного ПЗ.

2. Розуміння того, навіщо фізично розділяти інтерфейс (.h) та саму реалізацію коду (.cpp).
3. Здобуття навички створення власних статичних бібліотек (.a).
4. Розуміння процесу лінкування та вміння самостійно виправляти класичну помилку undefined reference.
5. Практика використання оператора множинного вибору switch-case для створення інтерактивного меню.
6. Засвоєння багато альтернативного розгалуження if - else if для обробки складних фінансових умов.
7. Знайомство з низькорівневим програмуванням через використання побітових операцій та масок.
8. Навичка застосування тернарного оператора ? : для написання чистого та лаконічного коду.
9. Усвідомлення важливості перевірки області допустимих значень (ОДЗ) та захисту програми від «падінь».
10. Опанування концепції модульного тестування (Unit Testing), що є обов’язковою вимогою до сучасних розробників.
11. Вміння складати тест-кейси з чітким визначенням вхідних даних та очікуваних результатів.
12. Практичний досвід проведення системного мануального тестування готового додатка.
13. Глибоке розуміння того, чому комп'ютерна математика відрізняється від звичайної (проблема зберігання чисел типу double).
14. Навичка правильного інженерного порівняння дійсних чисел із використанням математичної похибки та модуля abs().
15. Практика організації безкінечних циклів while(true) із безпечним механізмом виходу.
16. Вміння обробляти некоректний ввід від користувача (гілка default).
17. Знайомство з керуючими символами, такими як \a, для взаємодії з апаратним забезпеченням (системний звук помилки).
18. Розуміння конфліктів кодування (UTF-8 проти CP1251) та навичка усунення «кракозябрів» у консолі.
19. Практика візуального форматування виводу за допомогою бібліотеки <iomanip> (зокрема маніпулятора setw).
20. Розуміння механізму передачі параметрів за посиланням (&), щоб функція могла змінювати кілька змінних одночасно.

21. Розвиток базової гігієни роботи з системою контролю версій Git.
22. Формування звички не комітити в репозиторій скомпільовані бінарники (.exe, .o).
23. Вміння писати чіткі повідомлення для комітів, які відображають суть змін.
24. Розвиток уміння читати логи IDE та знаходити причину поломки замість сліпого натискання кнопки Run.
25. Практика «ручного» дебаггінгу через виведення проміжних станів змінних у консоль).
26. Досвід одночасного керування кількома пов'язаними проектами (бібліотека та драйвер) в одному середовищі Code::Blocks.
27. Усвідомлення принципу єдиної відповідальності: математична функція тільки рахує, драйвер — тільки виводить текст.
28. Розвиток навички перетворення сухого академічного тексту методички на працюючий програмний алгоритм.
29. Здобуття досвіду проектування архітектури програми до написання першого рядка коду.
30. Тренування уважності до деталей (наприклад, виявлення невидимих нерозривних пробілів, які ламають компіляцію).
31. Розуміння того, як налаштовуються шляхи пошуку (Search directories) для заголовкових файлів.
32. Закріплення знань про використання оператора return для дострокового завершення роботи функції при некоректних даних.
33. Робота із символьними типами даних (char) при обробці відповідей користувача (Y/N, j/J).
34. Розвиток алгоритмічного мислення при роботі з оператором побітового зсуву вправо (>>).
35. Розуміння використання логічного «І» (&) для перевірки конкретного біта числа.
36. Усвідомлення важливості правильного вибору типу даних (використання int для лічильників і розмірів одягу замість універсального double).
37. Досвід оформлення професійної технічної документації та прототипів функцій.
38. Знайомство з концепцією регресійного тестування (необхідність запускати всі тести після внесення будь-яких змін).
39. Підготовка до технічних співбесід, де часто запитують про різницю

між локальними #include "..." та системними #include <...>.

40. Поповнення власного профілю на GitHub реальним багатомодульним проектом.
41. Засвоєння базових принципів створення дружнього інтерфейсу консольного додатка (підказки, чіткі меню).
42. Розвиток терпіння та витримки при пошуку неочевидних багів у робочому середовищі.
43. Чітке розуміння того, що компіляція та лінкування — це два абсолютно різні етапи збірки програми.
44. Вміння ізолювати проблему замість того, щоб переписувати весь код з нуля.
45. Відмова від глобальних змінних на користь безпечної передачі даних через параметри функцій.
46. Навичка правильного іменування функцій (наприклад, task9_1_discount), що робить код самодокументованим.
47. Досвід роботи з винятками (try-catch) під час виклику старих функцій (як s_calculation).
48. Усвідомлення того, що код пишеться не лише для машини, але й для інших програмістів (чи для себе в майбутньому).
49. Отримання досвіду здачі повноцінного програмного продукту, що відповідає жорстким вимогам «замовника».
50. Розуміння того, як налаштовуються шляхи пошуку (Search directories) для заголовкових файлів.

Висновок: У ході виконання лабораторної роботи №9 було поглиблено практичні навички розробки багатомодульного програмного забезпечення мовою C++ із використанням статичних бібліотек. Основну увагу було приділено реалізації алгоритмів із розгалуженням, циклічних процесів та низькорівневих бітових операцій.